

Feature Engineering and Logistic Regression

1. Label Encoding / Ordinal Encoding

Definition

Label Encoding is a technique that converts each categorical value in a column into a unique integer (e.g., 0, 1, 2, 3...) based on alphabetical order or a user-defined order.

Explanation

When categories have an inherent order, ranking, or hierarchy, we use this method. It translates text categories into numbers so that mathematical models can understand the relative distance or order between those categories.

Example

Imagine an Education_Level column with values: High School, Undergraduate, Postgraduate, PhD.

- High School -> 0
- Undergraduate -> 1
- Postgraduate -> 2
- PhD -> 3

What Kind of Data We Use This On

- **Ordinal Data:** Categorical data where the order or rank strictly matters.
- *Examples:* Clothes sizes (S, M, L, XL), Customer Satisfaction Ratings (Poor, Average, Good, Excellent), Company Hierarchy (Junior, Mid-level, Senior, Executive).

How to Recognize When to Use It

Ask yourself: "**Does it make logical sense to say Category A is greater than Category B?**" If yes (e.g., PhD > High School), use Label/Ordinal Encoding.

2. One-Hot Encoding (OHE)

Definition

One-Hot Encoding splits a single categorical column into multiple new binary columns (dummy variables), where each column represents a unique category containing only 1 (True) or 0 (False).

Explanation

Most machine learning algorithms look at numbers and think higher numbers mean higher value or weight. If we give arbitrary numbers to unordered categories, the model gets confused. OHE fixes this by creating a checklist of columns, ensuring all categories are treated equally with no artificial ordering.

Example

Imagine a Marital_Status column with values: Single, Married, Divorced.

OHE will transform this into three distinct columns:

Original Column	Is_Single	Is_Married	Is_Divorced
Single	1	0	0
Married	0	1	0
Divorced	0	0	1

What Kind of Data We Use This On

- **Nominal Data (Low to Moderate Cardinality):** Categorical data where there is absolutely no rank, order, or relationship between the categories, and the total number of unique categories is relatively small (under 15-20 unique values).
- *Examples:* Gender (Male, Female, Other), Colors (Red, Blue, Green), Country of Residence (India, USA, UK).

How to Recognize When to Use It

Ask yourself two questions:

1. **"Is there any ranking or order between these categories?"** -> No.

2. "Are there only a few unique categories in this column?" -> Yes.

If both are true, One-Hot Encoding is your best choice. *(Remember to drop the first column using `drop_first=True` to avoid the dummy variable trap!)*

3. Frequency Encoding (Count Encoding)

Definition

Frequency Encoding replaces each categorical value in a column with its frequency representation, either the absolute count or the percentage (proportion) of times it appears in the entire dataset.

Explanation

Sometimes, the popularity or occurrence rate of a category holds predictive power. Instead of inventing arbitrary numbers or expanding the dataset horizontally with OHE, we look at the dataset as a whole and map the category directly to its global probability or distribution count.

Example

Suppose you have a City column in a dataset of 100 rows.

- Mumbai appears 50 times -> Replaced by **0.50** (or 50)
- Delhi appears 30 times -> Replaced by **0.30** (or 30)
- Kolkata appears 20 times -> Replaced by **0.20** (or 20)

What Kind of Data We Use This On

- **Nominal Data with High Cardinality:** Categorical data with zero natural order, but containing a very large number of unique values (e.g., dozens, hundreds, or thousands of unique categories).
- *Examples:* ZIP/Postal Codes, Website URL Referrers, Vehicle Model Names, IP Addresses.

How to Recognize When to Use It

Ask yourself: "**Does this column have too many unique categories for One-Hot Encoding, and could the frequency or popularity of the category be helpful for my model?**" If yes, use Frequency Encoding.

4. Target Encoding (Mean Encoding)

Definition

Target Encoding replaces each categorical value with the average (mean) value of the **Target Variable** for that specific category.

Explanation

This is a highly supervised encoding method. Instead of looking just at the feature itself, it looks at how the feature behaves relative to what you are trying to predict. In classification tasks, the category is replaced by the probability of the target being 1. In regression tasks, it is replaced by the average target value.

Example

Imagine predicting if a user will buy a product (Target: 1 for bought, 0 for skipped) based on their City:

- Out of 10 people from Mumbai, 8 people bought the product. The mean target value for Mumbai is $8 / 10 = 0.8$. All rows with "Mumbai" are replaced by 0.8.
- Out of 10 people from Delhi, 2 people bought the product. The mean target value for Delhi is $2 / 10 = 0.2$. All rows with "Delhi" are replaced by 0.2.

What Kind of Data We Use This On

- **High Cardinality Nominal Data for Predictive Modeling:** Used when a column has too many categories for OHE, and you want a very strong, direct numerical relationship mapped between that feature and your final target variable.
- *Examples:* Product Categories in e-commerce, User IDs, Detailed Job Titles.

How to Recognize When to Use It

Ask yourself: **"Do I have a high-cardinality categorical feature that strongly impacts my target variable, and am I using cross-validation or smoothing techniques to prevent data leakage?"** If yes, Target Encoding will give your machine learning model a massive performance boost.

Master Cheat Sheet: Which Encoding Do I Choose?		
Is there a natural order/rank?	Number of unique categories?	Recommended Encoding Method
Yes (e.g., Low, Medium, High)	Any amount	Label / Ordinal Encoding
No (e.g., Red, Green, Blue)	Low (under 10-15 categories)	One-Hot Encoding
No (e.g., 500 different ZIP codes)	High (Count matters to the model)	Frequency Encoding
No (e.g., 1000 different Product IDs)	High (Strongly tied to the target)	Target Encoding

1. Logistic Regression

Definition

Logistic Regression is a supervised learning algorithm used to predict the probability of a binary target variable (a variable that has exactly two possible outcomes, such as Yes/No, 0/1, or Success/Failure).

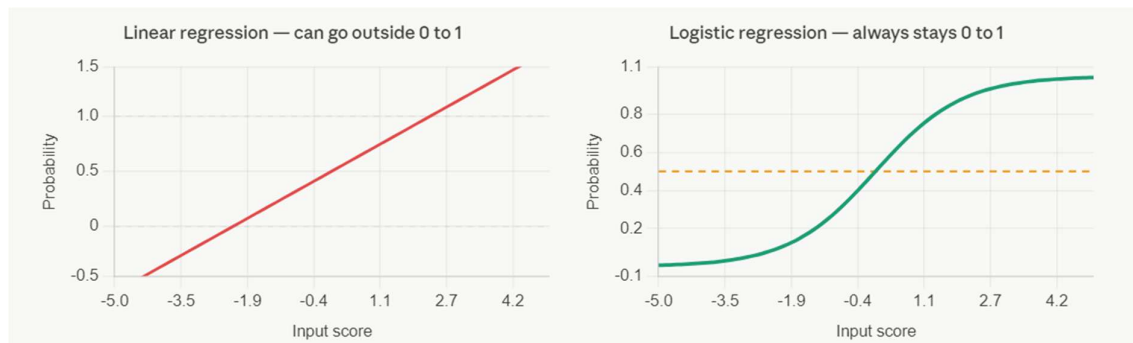
What is Logistic Regression?

Despite having "regression" in the name, it's actually a **classification** algorithm. Its job is to answer YES or NO, to split data into two categories.

"Is this email spam or not?" "Does this patient have diabetes or not?" "Will this customer churn or not?"

The Core Idea: Why Not Just Use Linear Regression?

Linear regression draws a straight line that can predict any value, including 1.8 or -0.3, which make no sense as probabilities. Logistic regression solves this by squashing any value through a special **S-shaped curve** (the sigmoid function) that always outputs between 0 and 1.



The orange dashed line is the **decision boundary at 0.5**. Above it → Class 1 (YES). Below it → Class 0 (NO).

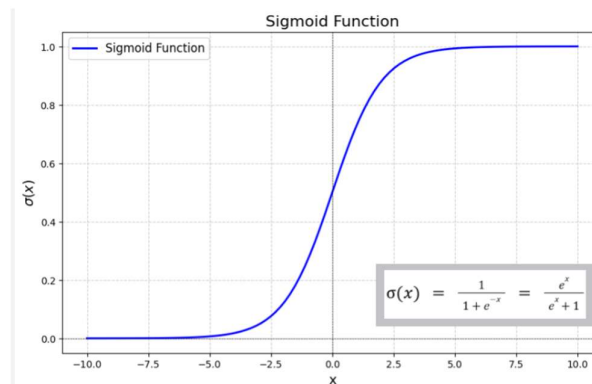
The Analogy: Doctor's Diagnosis

Think of a doctor reviewing a patient's test results: blood sugar level, age, and BMI. The doctor makes one decision - "Does this patient have diabetes?"

That's exactly what logistic regression does. It takes all the input values, runs them through a formula, and outputs a single probability - like "78% chance of diabetes." If probability ≥ 0.5 , predict YES. Otherwise, NO.

Sigmoid Function

The **sigmoid function** is a mathematical function with a characteristic "S"-shaped curve. It maps any real-valued number to a value between (0) and (1) . Because of this, it is highly useful in machine learning and statistics for converting raw outputs into probabilities



The following interactive stepper walks through each stage of the algorithm.

Step 1 — Collect input features

Each patient comes with measurable attributes. These are called **features** — the inputs your model will learn from.



Step 2 — Compute a weighted sum (z)

Multiply each feature by a learned **weight**, add them all up, and add a bias. This is identical to linear regression. The weight tells the model how important that feature is.

$$z = w_1 \times \text{age} + w_2 \times \text{blood_sugar} + w_3 \times \text{bmi} + \text{bias} = 0.04 \times 45 + 0.02 \times 180 + 0.05 \times 28.3 + (-6.2) = 1.8 + 3.6 + 1.4 - 6.2 = 0.6$$

Step 3 — Apply the sigmoid function

The raw score z (can be any number) is squeezed through the **sigmoid** to give a probability between 0 and 1. Higher $z \rightarrow$ probability closer to 1.

$\text{sigmoid}(z) = 1 / (1 + e^{-z})$ $\text{sigmoid}(0.6) = 1 / (1 + e^{-0.6}) \approx 0.645$ Interpretation: 64.5% chance of diabetes

Step 4 — Apply the decision threshold

Compare the probability to a threshold (default 0.5). Above it $\rightarrow$ predict YES, below $\rightarrow$ predict NO. You can tune this threshold based on whether false negatives or false positives are more costly.

$P = 0.645 \geq 0.5$ threshold $\rightarrow$ Predict: DIABETES

Step 5 — Training with gradient descent

Initially the weights are random, so predictions are wrong. Training adjusts the weights thousands of times to minimize **log-loss** (the error). Each iteration makes the model slightly more accurate.

Error = actual_label - predicted_probability Update = weight - learning_rate $\times$ error $\cup$ Repeat until error stops decreasing

Example

Predicting whether a bank customer will default on a loan (1) or not default (0) based on their Income and Credit Score. If the model outputs a score of 0.85, it implies there is an 85% probability that the customer will default.

How to Recognize When to Use It

Ask yourself: "**Is my target variable categorical with exactly two distinct classes (Binary Classification)?**" If yes, Logistic Regression should serve as your baseline machine learning model.

CLASSIFICATION METRICS (THE EVALUATION)

Once a model generates predictions, we must assess its performance. Relying solely on overall "Accuracy" can be highly misleading, especially when dealing with imbalanced datasets. Therefore, we use a suite of evaluation metrics.

1. Confusion Matrix

Definition

A Confusion Matrix is a tabular layout that visualizes the performance of a classification algorithm by systematically comparing the actual target values against the values predicted by the model.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Explanation

It is a 2x2 grid (for binary classification) that explicitly reveals where your model was correct and where it got "confused." It breaks down the predictions into four core metrics:

1. **True Positive (TP):** The actual class was 1, and the model correctly predicted 1.
2. **True Negative (TN):** The actual class was 0, and the model correctly predicted 0.
3. **False Positive (FP) [Type I Error]:** The actual class was 0, but the model incorrectly predicted 1. (A False Alarm).
4. **False Negative (FN) [Type II Error]:** The actual class was 1, but the model incorrectly predicted 0. (A Missed Opportunity).

3. Precision

Definition

Precision is the ratio of correctly predicted positive observations to the total predicted positive observations made by the model.

Formula

$$Precision = \frac{TP}{TP + FP}$$

Explanation

Precision answers the question: **"Out of all the cases the model flagged as POSITIVE (1), how many were actually positive?"** We optimize for Precision when the cost of a False Positive (FP) is exceptionally high.

Example (When to focus on Precision)

Spam Email Detection: If a vital work email (Actual: Not Spam - 0) is incorrectly flagged as Spam (Predicted: Spam - 1), it results in a critical False Positive. The user might miss urgent information. Therefore, we demand high Precision so that when a mail is marked as spam, the model is highly certain.

4. Recall (Sensitivity)

Definition

Recall is the ratio of correctly predicted positive observations to all actual observations belonging to the positive class.

Formula

$$recall = \frac{TP}{TP + FN}$$

Explanation

Recall answers the question: **"Out of all the actual positive cases present in the dataset, how many did the model successfully find and capture?"** We optimize for Recall when missing a positive case (a False Negative) carries dangerous consequences.

Example

Cancer Detection: If a patient has cancer (Actual: 1) but the model screens them and predicts they are healthy (Predicted: 0), this creates a catastrophic False Negative. The patient goes untreated, risking their life. We need a high Recall model to ensure no positive case goes undetected.

5. F1-Score

The F1 score is a machine learning metric used to evaluate the performance of a classification model. It combines precision and recall into a single value, making it especially useful when dealing with imbalanced datasets where one class outnumbers the other.

To understand it, we break down its components:

- **Precision:** The percentage of positive predictions that were actually correct.
- **Recall:** The percentage of actual positive cases that the model successfully identified.

The Formula

$$\text{F1 Score} = \frac{2}{\left(\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}} \right)}$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why use the F1 score?

- **Handles Imbalanced Data:** If you have 99% of one class and 1% of another (like in fraud detection), a model can get 99% accuracy simply by guessing the majority class every time. The F1 score ignores true negatives and focuses strictly on the minority class, providing a more truthful measure of success.
- **Penalizes Lopsided Models:** The harmonic mean heavily penalizes models that favor either precision or recall too much. A high F1 score requires **both** metrics to be strong

For the example of Logistic Regression taken in ipynb file:

1. The Machine Learning Pipeline Order

When building a classification model, the standard and correct sequence of operations is:

- **Step 1: Data Cleaning:** Handle missing values (NaN) and remove duplicates before any transformation.
- **Step 2: Categorical Encoding:** Convert text features into numbers using One-Hot Encoding (`pd.get_dummies`) because mathematical models cannot process text.
- **Step 3: Train-Test Split:** Divide the dataset (e.g., 80% Train, 20% Test) **before** scaling to prevent **Data Leakage** (test data info leaking into training).
- **Step 4: Feature Scaling (Standardization):** Scale features using `StandardScaler` (mean=0, variance=1) so features with large ranges (e.g., Monthly Charges) don't dominate the model weights.
 - Apply `.fit_transform()` on Train data, but *only* `.transform()` on Test data.
- **Step 5: Model Training & Prediction:** Train using `model.fit()` and generate predictions using `model.predict()`.
- **Step 6: Evaluation:** Analyze performance using Confusion Matrix, Precision, and Recall.

2. Confusion Matrix Deconstruction

For a binary classification problem (where 0 = Stayed, 1 = Churn):

Based on your model's specific output:

`[[91 23] <- [TN FP]`

`[37 49]] <- [FN TP]`

- **True Negative (TN = 91):** Customers who actually stayed, and the model correctly predicted they would stay.

- **True Positive (TP = 49):** Customers who actually churned, and the model correctly caught them.
- **False Positive (FP = 23) [Type I Error]:** Customers who stayed, but the model wrongly flagged them as "will churn." (*Business Cost: Wasted retention discounts*).
- **False Negative (FN = 37) [Type II Error]:** Customers who actually churned, but the model wrongly predicted they would stay. (*Business Cost: High. Company loses these customers completely without a chance to save them*).

3. Core Evaluation Metrics & Formulas

A. Precision (Quality of Positive Predictions)

- **Formula:**

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- **Your Score:** $49 / (49 + 23) = 0.68 \rightarrow 68\%$
- **Interpretation:** Out of all the customers the model predicted *would* churn, 68% of them actually did. It measures how trustworthy the model's positive alarms are.

B. Recall / Sensitivity (Quantity of Positives Caught)

- **Formula:**

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

- **Your Score:** $49 / (49 + 37) = 0.57 \rightarrow 57\%$
- **Interpretation:** Out of all the actual churners present in the dataset, the model only successfully identified 57% of them. The remaining 43% sneaked past undetected.

C. F1-Score (The Balanced Metric)

- **Formula:**

$$\text{F1-Score} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

- **Your Score:** 0.62 for Class 1.
- **Interpretation:** The harmonic mean of Precision and Recall. It gives a single robust score to evaluate imbalanced datasets where you care about both false alarms and missed targets.

D. Support

- **Meaning:** The actual count of true instances for each class in the test subset. In your data, Support for 0 = 114 and Support for 1 = 86 (Total 200 test samples).

4. Business Justification Summary

- **Current Status:** The baseline Logistic Regression model achieves **70% overall accuracy**.
- **The Problem:** A **57% Recall** means the model is too conservative and misses 43% of churning customers.
- **Next Steps for Optimization:** To increase business ROI, the focus must be on improving **Recall** (by lowering the classification threshold or adding richer behavioral features) so the business can intercept more at-risk customers before they leave.